

In §19.3 we discuss how to proceed in the case of singular kernels, focusing largely on Fredholm equations (both first and second kinds). Singularities require special quadrature rules, but they are also sometimes blessings in disguise, since they can spoil a kernel's smoothing and make problems well-conditioned.

In §19.4 – §19.7 we face up to the issues of inverse problems. Section 19.4 is an introduction to this large subject.

We should note here that wavelet transforms, already discussed in §13.10, are applicable not only to data compression and signal processing, but can also be used to transform some classes of integral equations into sparse linear problems that allow fast solution. You may wish to review §13.10 as part of reading this chapter.

Some subjects, such as *integro-differential equations*, we must simply declare to be beyond our scope. For a review of methods for integro-differential equations, see Brunner [4].

It should go without saying that this one short chapter can only barely touch on a few of the most basic methods involved in this complicated subject.

CITED REFERENCES AND FURTHER READING:

- Delves, L.M., and Mohamed, J.L. 1985, *Computational Methods for Integral Equations* (Cambridge, UK: Cambridge University Press).[1]
 Linz, P. 1985, *Analytical and Numerical Methods for Volterra Equations* (Philadelphia: S.I.A.M.).[2]
 Atkinson, K.E. 1976, *A Survey of Numerical Methods for the Solution of Fredholm Integral Equations of the Second Kind* (Philadelphia: S.I.A.M.).[3]
 Brunner, H. 1988, in *Numerical Analysis 1987*, Pitman Research Notes in Mathematics vol. 170, D.F. Griffiths and G.A. Watson, eds. (Harlow, Essex, UK: Longman Scientific and Technical), pp. 18–38.[4]
 Smithies, F. 1958, *Integral Equations* (Cambridge, UK: Cambridge University Press).
 Kanwal, R.P. 1971, *Linear Integral Equations* (New York: Academic Press).
 Green, C.D. 1969, *Integral Equation Methods* (New York: Barnes & Noble).

19.1 Fredholm Equations of the Second Kind

We desire a numerical solution for $f(t)$ in the equation

$$f(t) = \lambda \int_a^b K(t, s) f(s) ds + g(t) \quad (19.1.1)$$

The method we describe, a very basic one, is called the *Nystrom method*. It requires the choice of some approximate *quadrature rule*:

$$\int_a^b y(s) ds = \sum_{j=0}^{N-1} w_j y(s_j) \quad (19.1.2)$$

Here the set $\{w_j\}$ are the weights of the quadrature rule, while the N points $\{s_j\}$ are the abscissas.

What quadrature rule should we use? It is certainly possible to solve integral equations with low-order quadrature rules like the extended trapezoidal or Simpson's

rules. We will see, however, that the solution method involves $O(N^3)$ operations, and so the most efficient methods tend to use high-order quadrature rules to keep N as small as possible. For smooth, nonsingular problems, nothing beats Gaussian quadrature (e.g., Gauss-Legendre quadrature, §4.6). (For nonsmooth or singular kernels, see §19.3.)

Delves and Mohamed [1] investigated methods more complicated than the Nystrom method. For straightforward Fredholm equations of the second kind, they concluded “... the clear winner of this contest has been the Nystrom routine ... with the N -point Gauss-Legendre rule. This routine is extremely simple. ... Such results are enough to make a numerical analyst weep.”

If we apply the quadrature rule (19.1.2) to equation (19.1.1), we get

$$f(t) = \lambda \sum_{j=0}^{N-1} w_j K(t, s_j) f(s_j) + g(t) \quad (19.1.3)$$

Evaluate equation (19.1.3) at the quadrature points:

$$f(t_i) = \lambda \sum_{j=0}^{N-1} w_j K(t_i, s_j) f(s_j) + g(t_i) \quad (19.1.4)$$

Let f_i be the vector $f(t_i)$, g_i the vector $g(t_i)$, K_{ij} the matrix $K(t_i, s_j)$, and define

$$\tilde{K}_{ij} = K_{ij} w_j \quad (19.1.5)$$

Then, in matrix notation, equation (19.1.4) becomes

$$(\mathbf{1} - \lambda \tilde{\mathbf{K}}) \cdot \mathbf{f} = \mathbf{g} \quad (19.1.6)$$

This is a set of N linear algebraic equations in N unknowns that can be solved by standard triangular decomposition techniques (§2.3) — that is where the $O(N^3)$ operations count comes in. The solution is usually well-conditioned, unless λ is very close to an eigenvalue.

Having obtained the solution at the quadrature points $\{t_i\}$, how do you get the solution at some other point t ? You do *not* simply use polynomial interpolation. This destroys all the accuracy you have worked so hard to achieve. Nystrom’s key observation was that you should use equation (19.1.3) as an interpolatory formula, maintaining the accuracy of the solution.

Our routine for solving linear Fredholm equations of the second kind is coded as the object `Fred2`. The constructor sets up equation (19.1.6) and then solves it by LU decomposition with `LUdcmp`. The Gauss-Legendre quadrature is implemented by first getting the weights and abscissas with a call to `gauleg`. The routine `Fred2` requires that you provide an external function or functor that returns $g(t)$ and another that returns λK_{ij} . It then computes the solution f at the quadrature points in the member variable `f`. It also stores the quadrature points and weights. These are used by the member function `fredin` to carry out the Nystrom interpolation of equation (19.1.3) and return the value of f at any point in the interval $[a, b]$.

To be sure the usage is clear, here is the calling sequence when you have coded the external routines as functions:

```

Doub g(const Doub t) { ... }
Doub ak(const Doub t, const Doub s) { ... }
...
Fred2<Doub (Doub), Doub (Doub,Doub)> fred2(a,b,n,g,ak);
Doub ans=fred2.fredin(x);

```

If the external routines are functors Gfunc and Kernel, say, then the declarations are

```

Gfunc g;                                This could have arguments if you like.
Kernel ak;
Fred2<Gfunc, Kernel> fred2(a,b,n,g,ak);

```

Here is the routine:

```

template <class G, class K>                                fred2.h
struct Fred2 {
Solves a linear Fredholm equation of the second kind.
    const Doub a,b;
    const Int n;
    G &g;
    K &ak;
    VecDoub t,f,w;
    Fred2(const Doub aa, const Doub bb, const Int nn, G &gg, K &akk) :
        a(aa), b(bb), n(nn), g(gg), ak(akk), t(n), f(n), w(n)
    Quantities a and b are input as the limits of integration. The quantity n is the number of
    points to use in the Gaussian quadrature. g and ak are user-supplied functions or functors
    that respectively return  $g(t)$  and  $\lambda K(t,s)$ . This constructor computes arrays t[0..n-1]
    and f[0..n-1] containing the abscissas  $t_i$  of the Gaussian quadrature and the solution  $f$ 
    at these abscissas. Also computed is the array w[0..n-1] of Gaussian weights for use with
    the Nystrom interpolation routine fredin.
    {
        MatDoub omk(n,n);
        gauleg(a,b,t,w);                                Replace gauleg with another routine if not using
        for (Int i=0;i<n;i++) {                            Gauss-Legendre quadrature.
            for (Int j=0;j<n;j++)                            Form  $\mathbf{1} - \lambda \tilde{\mathbf{K}}$ .
                omk[i][j]=Doub(i == j)-ak(t[i],t[j])*w[j];
            f[i]=g(t[i]);
        }
        LUdcmp alu(omk);                                Solve linear equations.
        alu.solve(f,f);
    }

    Doub fredin(const Doub x)
    Given arrays t[0..n-1] and w[0..n-1] containing the abscissas and weights of the Gauss-
    ian quadrature, and given the solution array f[0..n-1], this function returns the value of
     $f$  at x using the Nystrom interpolation formula.
    {
        Doub sum=0.0;
        for (Int i=0;i<n;i++) sum += ak(x,t[i])*w[i]*f[i];
        return g(x)+sum;
    }
};

```

One disadvantage of a method based on Gaussian quadrature is that there is no simple way to obtain an estimate of the error in the result. The best practical method is to increase N by 50%, say, and treat the difference between the two estimates as a conservative estimate of the error in the result obtained with the larger value of N .

Turn now to solutions of the homogeneous equation. If we set $\lambda = 1/\sigma$ and $g = 0$, then equation (19.1.6) becomes a standard eigenvalue equation,